

Project idea 1:

Food allergy analyzer:

One project I have thought about pursuing is a food allergy analyzer application that allows users to input their food allergies and scan a product barcode to quickly determine whether the item is safe to consume. I created an app prototype last semester for a marketplace style listing app that helps people reduce food waste, and a part of that app was a section where people input their allergies when searching to automatically filter out listings that have those allergies. Though I don't think it's feasible to make the whole app, I thought I could do that one section. Here is the prototype if you want to review it: <https://robust-flop-68190342.figma.site/> I would be interested in actually building a similar app sometime for use in my community, and thought I could use this class to start.

The primary users would be individuals with food allergies or dietary restrictions, while secondary users could include parents of children with allergies, healthcare professionals, or caregivers. The main task of the application is to help users make safer and faster food decisions by automatically analyzing ingredient and allergen information from ingredient listings. The primary interaction loop would involve the user inputting their allergens, the system retrieving product information from a database of all listings, analyzing the ingredients against the user's allergy profile, and then returning a clear safety result such as "safe," "caution," or "unsafe." The application could be built as a mobile-friendly Flask web application with a simple dashboard. The project would use food product and ingredient data from APIs such as the [Open Food Facts API](#), which contains barcode-searchable nutrition and allergen information, and then also apply it to scan from the listed ingredients in the listings. Data would primarily be retrieved in JSON format and processed using Python libraries such as requests and pandas. Ingredient lists would need to be cleaned and standardized to detect allergens and hidden ingredient names (for example, recognizing that "casein" is a dairy ingredient). The results would then be presented visually through color-coded alerts, ingredient highlighting, and safety indicators to help users quickly understand whether the listing matches their dietary needs.

I would want here to be a search bar that allows people to add additional allergens by name, ie peppers, onions, or strawberries.

I think I could do it myself, I just need to brush up on cleaning and standardizing data so that the search exclusion for specific pieces works. I would like to implement this for my food waste app, but also thought it could be a stand alone, and include a camera function that allows people to scan barcodes while shopping to search for allergens. Flask was the last assignment we did last semester and I enjoyed that one, so I think I could work with it.

Project idea 2:

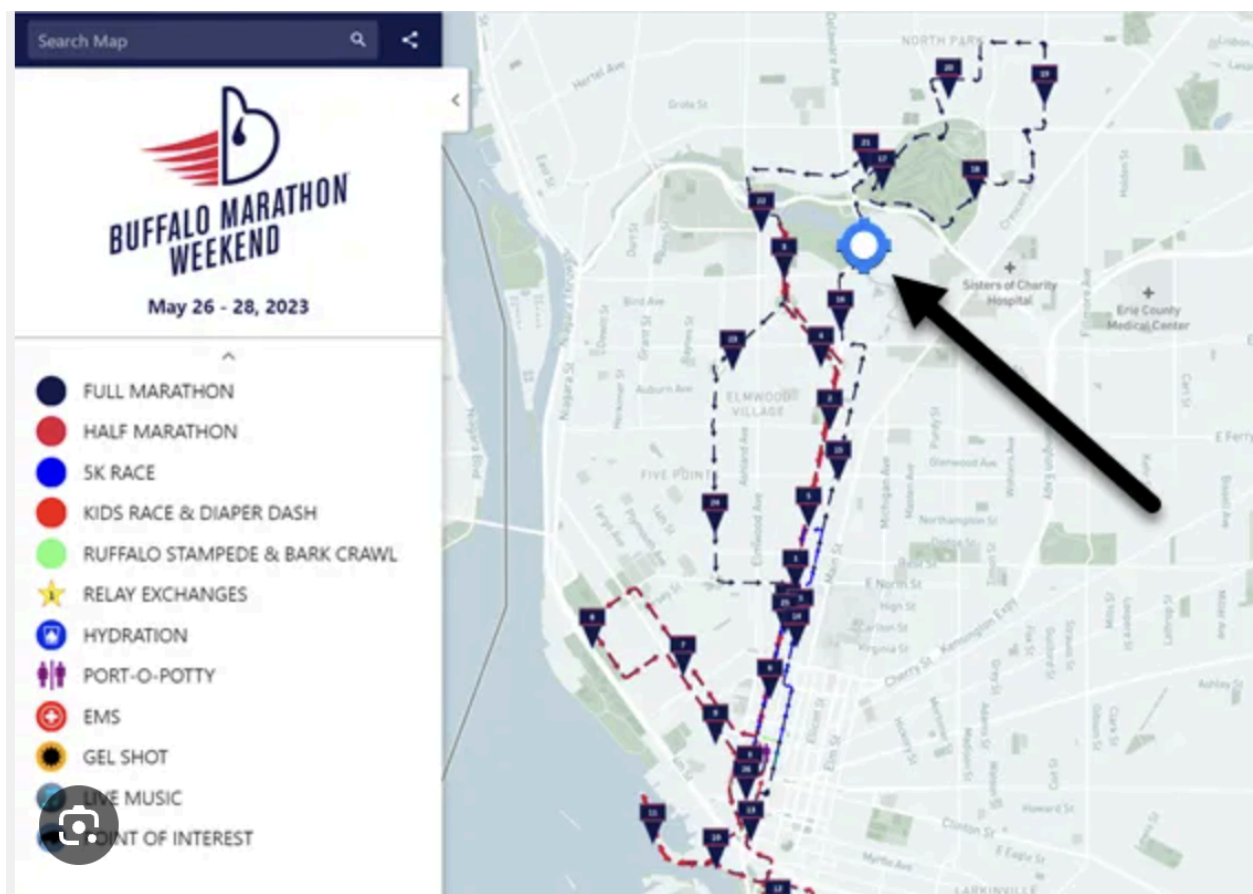
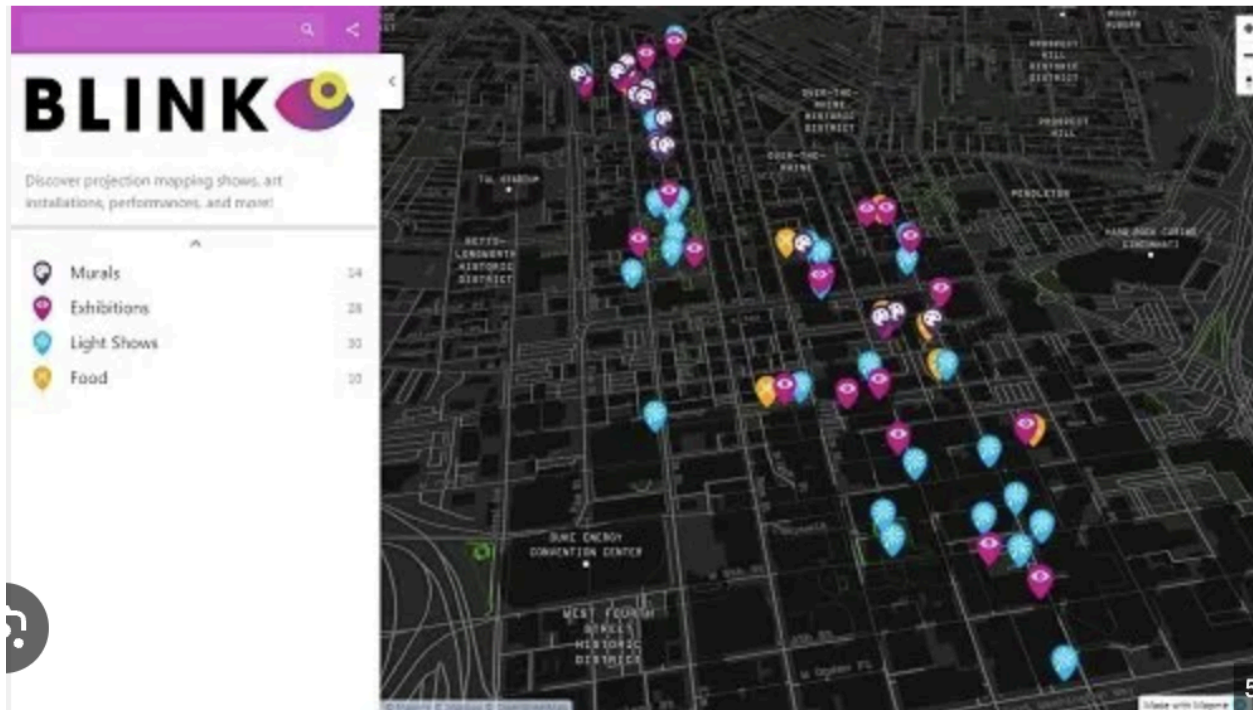
Ticket analyzer:

Another possible idea is an interactive Kansas City event application that helps users find concerts, festivals, museums, restaurants, coffee shops, and local events in the Kansas City area. Kansas city has a lot of cool things going on, but not a lot of pages or blogs dedicated to talking about them. The primary users would be local residents, college students, and tourists looking for entertainment or activities, while secondary users could include local businesses and event organizers who benefit from increased visibility.

The main task of the application would be to simplify event discovery by combining information from multiple sources into one searchable and visual platform. Users would interact with the application by selecting interests, searching for activities, filtering by date or price, and exploring events through an interactive map. The interaction loop would involve searching for events, viewing recommendations, saving favorites, and exploring nearby activities. A web-based interface created with Flask and JavaScript could provide an interactive map and dashboard. The project would use event and location data gathered from APIs such as the [Ticketmaster Discovery API](#) and possibly Eventbrite or OpenStreetMap datasets. Data would likely be retrieved in JSON format and could include event names, categories, ticket prices, locations, and dates. Processing would involve cleaning duplicate event entries, categorizing events, geocoding addresses, and filtering results based on user preferences such as budget, interests, or distance. Python libraries such as pandas and geopandas could be used for processing and visualization. Results would be presented through interactive GIS maps, event timelines, recommendation lists, and visual clusters showing popular activity areas around Kansas City. This project would demonstrate data collection, API integration, geospatial visualization, user interaction, and front-end design in a practical and visually engaging way.

I think the hardest part for me would be the map integration, but KC is a very wide area so I would like people to be able to visualize which neighborhood they would want to be in.

I like how some of the sample maps below are displayed.



Project idea 3:

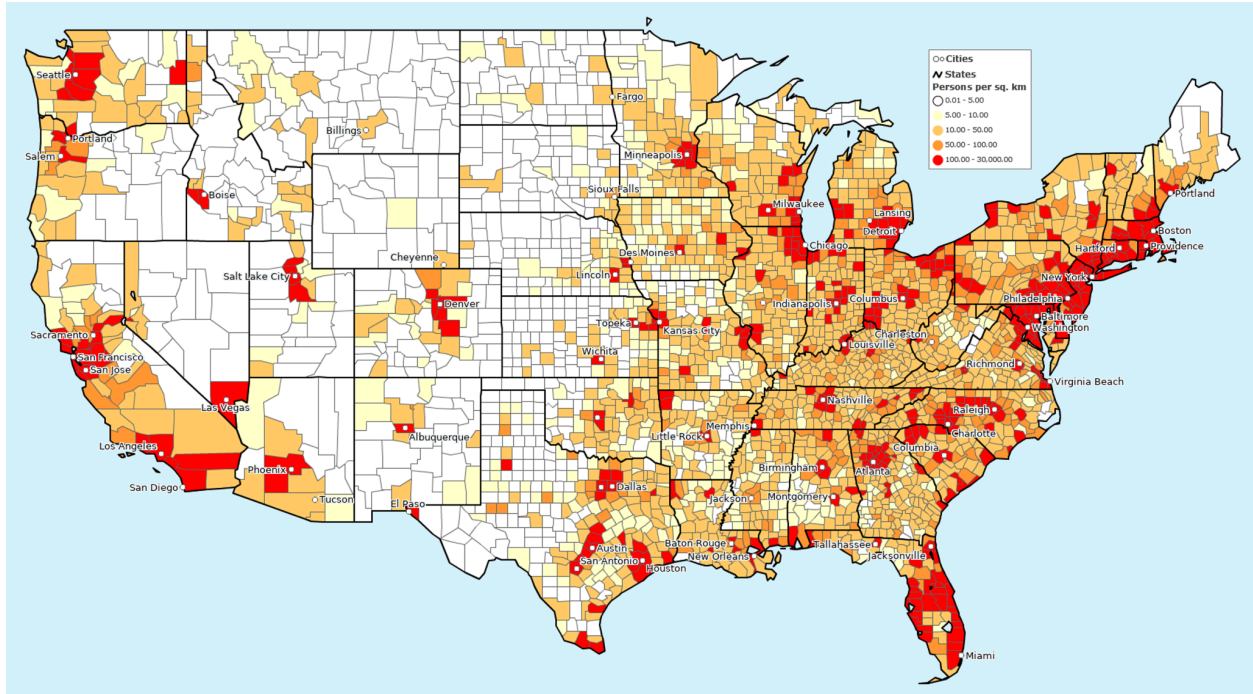
Where not to go:

I strongly dislike vacationing during times when there are a lot of tourists in the place where I am traveling. Also, with KC being a destination of one of the upcoming world cups and it seems like each news source is predicting a different amount of travelers, I was wondering if it was possible to create an app that synchronized all airline and travel information to allow predictions as to the volume of consumers in a given area. When researching, I discovered that it would be difficult to gather information from airlines directly, as that is proprietary information, but instead of using actual ticket sales numbers, the application could combine multiple public datasets to estimate travel demand. For example, APIs such as [Amadeus for Developers](#) provide flight and travel data, while Google Trends, hotel price APIs, Ticketmaster events, weather, and seasonal tourism statistics could act as indirect indicators of visitor volume. I could develop a Flask web application with interactive charts and maps.

The system could process flight frequency, ticket price trends, major local events, and seasonal travel patterns to estimate expected tourist density in a location. Python libraries such as pandas, scikit-learn, Plotly, and geopandas could be used to process and visualize the data. Results could be displayed through interactive heat maps, crowd-level gauges, trend graphs, and destination rankings showing which locations are expected to be busiest during certain times of year.

The primary users could be travelers trying to avoid overcrowded destinations, local businesses planning staffing, or tourists looking for cheaper and less busy travel periods. Secondary users could include hotels, restaurants, or city tourism organizations. The core task would be helping users predict how crowded a destination may be during a given week or month. The interaction loop could involve a user selecting a destination and date range, the system collecting travel-related data from APIs and datasets, processing trends and demand indicators, and then displaying a predicted “tourism intensity” score. This would help people know where to travel - from a marketing perspective I think I would call it “Off-season” and ideally if possible the user could see a map of the US, set date ranges at the top of the map, and we would get a heat map of expected people in certain areas during that time.

I would like to preface that I have no idea as to how difficult this would be for a novice Python coder like myself, I just think it is a really cool idea.



Like this map, but only showcasing tourists, as it is expected that places like New York obviously have more population on the regular.

Project idea 4:

Variable weather widget:

This project idea is most likely the one I have the most confidence in actually being able to execute as well as something that could actually be applied at my workplace. At work we have an internal internet that employees use as well as public facing digital displays for wayfinding. The intranet has a current weather widget that I think could be greatly improved upon based on user complaints and my boss has requested a weather widget be applied to our public displays since a lot of waiting rooms and patient areas have no windows.

The weather dashboard widget would display current weather conditions and forecasts while dynamically changing the background visuals based on the weather. The primary users would be employees in an office/healthcare setting, while secondary users could include workplace administrators or visitors viewing the display in shared spaces such as lobbies or break rooms (if I get it to work well enough to be public facing). The goal of the application would be to provide useful real-time weather information in a visually engaging and easy-to-read format. Users would primarily interact with the system by viewing the dashboard, while administrators could potentially customize locations, themes, or alerts based on our multiple campuses. The interaction loop would involve the application continuously retrieving updated weather data, processing weather conditions, and automatically updating the interface visuals and forecasts.

The project could be implemented as a Flask web application using HTML, CSS, and JavaScript, allowing it to run on office TVs, kiosks, tablets, or internal web pages.

The project would use live weather data from APIs such as the [OpenWeather API](#) or [WeatherAPI.com](#). Data would be retrieved in JSON format and could include temperature, humidity, weather conditions, forecasts, and severe weather alerts (we get a lot of severe weather alerts in spring and it is an item of much concern for our clinical staff, so it would be great if it would display our custom messaging on whether or not to move patients during a Tornado Warning based on specific conditions or administrator push). The processing portion of the project would involve mapping weather conditions to different background images, themes, or animations, such as displaying bright visuals for sunny weather or darker themes during storms. We have a custom illustration style at the hospital, so I could easily make a deck of illustrations or even animations to display. Python libraries such as requests and pandas could be used for data retrieval and processing, while visualization would focus on interactive forecast cards, weather icons, animated transitions, and responsive layouts. Results would be presented through a clean graphical dashboard and forecast timelines. It would be cool then to see it realized and applied at my workplace.

Some quick sample dashboards I mocked up below:

